



INSTITUTO POLITÉCNICO DA UNIVERSIDADE KIMPA VITA

LICENCIATURA EM ENGENHARIA INFORMÁTICA

SISTEMA DE GESTÃO DE BASE DE DADOS

RELATÓRIO TÉCNICO

SISTEMA DE MONITORIZAÇÃO DE RUTURA DE MEDICAMENTOS ESSENCIAIS E SURTOS DE MALÁRIA NA PROVÍNCIA DO UÍGE

Arquitectura e Modelação Avançada em NoSQL

MAIO, 2026

INSTITUTO POLITÉCNICO DA UNIVERSIDADE KIMPA VITA

LICENCIATURA EM ENGENHARIA INFORMÁTICA

SISTEMA DE MONITORIZAÇÃO DE RUTURA DE MEDICAMENTOS ESSENCIAIS E SURTOS DE MALÁRIA NA PROVÍNCIA DO UÍGE

Unidade Curricular: SISTEMA DE GESTÃO DE BASE DE DADOS (SGBD II)

INTEGRANTES DO GRUPO:

Francisco Bráulio Figueiredo António

João José António Da Silva

Kumbi Manuel Domingos

Paulo Bunga Rodrigues

GRUPO 18 | 3º ANO | ANO LETIVO: 2025/2026

Orientado por:

Moyo Kanivengidio, Msc.

Conteúdo

RESUMO.....	5
ANÁLISE DO DOMÍNIO E PADRÕES DE ACESSO	6
Descrição do Subdomínio	6
Padrões de Acesso (Query-Driven Design)	6
JUSTIFICAÇÃO TECNOLÓGICA.....	7
Análise Comparativa de Modelos NoSQL.....	7
Justificação da Escolha: MongoDB.....	7
Adequação ao Modelo de Dados	7
Teorema CAP, PACELC e Configuração do MongoDB	8
Escalabilidade: Sharding Horizontal	8
MODELAÇÃO DE DADOS	9
Arquitectura das Coleções	9
Esquemas das Coleções Principais	9
Coleção: registos_stock (Série Temporal Principal)	9
Coleção: municipios (com índice Geoespacial)	10
Estratégias de Desnormalização	10
Estratégia de Indexação	10
IMPLEMENTAÇÃO E RESULTADOS	11
Ambiente de Implementação	11
Volume de Dados Gerados	11
Consultas Avançadas Implementadas	11
Excerto de Código Q5: Dashboard de Criticidade (Pipeline de Agregação) ...	12
DISCUSSÃO CRÍTICA.....	13
Limitações da Solução Actual	13
Escalabilidade: Cenário de Volume 100x	13
Comportamento em Falha de Rede (Partição).....	13
Melhorias Futuras	14
REFERÊNCIAS BIBLIOGRÁFICAS	15
ANEXO A (GUIA DE INSTALAÇÃO)	16
A.1 Instalação via Docker (Recomendada).....	16
A.2 Instalação Local — Linux (Ubuntu 22.04)	16

A.3 Verificações Rápidas no mongosh.....	17
--	----

RESUMO

A Província do Uíge, no norte de Angola, enfrenta desafios persistentes de saúde pública, nomeadamente a elevada incidência de malária por *Plasmodium falciparum* e a irregularidade no fornecimento de medicamentos antimaláricos às suas 16 unidades administrativas. A ausência de sistemas integrados de monitorização impede uma resposta atempada às ruturas de stock e dificulta a correlação entre a disponibilidade de medicamentos e os picos epidémicos.

Este trabalho propõe, implementa e avalia uma camada de persistência NoSQL orientada a documentos usando o MongoDB 8.2, capaz de monitorizar em tempo real os níveis de stock de 22 medicamentos essenciais em 52 unidades sanitárias, registar e correlacionar surtos de malária com ruturas de antimaláricos, e gerar alertas automáticos para os gestores distritais e para o Gabinete Provincial da Saúde

Componente	Descrição	Resultado
Base de Dados	MongoDB 8.2	Implementado
Volume de Dados	167.009 documentos em 6 coleções distintas	Excede requerido (100.000)
Índices	12 índices (geoespacial, composto, simples, único)	Otimizados
Queries	7 consultas avançadas (pipeline, geoNear, \$lookup duplo)	Executam < 3s
Cluster	Replica Set com failover automático e eleição por maioria	Demonstrado
Reprodutibilidade	Python (seeding) + JS (queries) + Docker Compose	Via README.md

O MongoDB demonstrou ser a escolha óptima para este domínio devido à sua flexibilidade de esquemas, suporte nativo de índices geoespaciais e poderosos pipelines de agregação. O sistema opera no quadrante CP do Teorema CAP, priorizando consistência e tolerância a partições, que é fundamental para dados de saúde pública onde a integridade é inegociável.

ANÁLISE DO DOMÍNIO E PADRÕES DE ACESSO

Descrição do Subdomínio

O sistema insere-se no subdomínio de Análise de Séries Temporais em Saúde Pública, enquadrado no ecossistema maior de gestão de saúde pública em Angola. Estudamos 16 municípios da província, numa população estimada de 1,7 milhões de habitantes (CENSO 2024), e regista anualmente entre 80 a 320 casos de malária por 1.000 habitantes, dependendo do município e da época do ano (predominantemente época chuvosa: fevereiro-abril e outubro-dezembro).

O sistema modela três entidades centrais: (1) as unidades sanitárias - hospitais provinciais, municipais, centros e postos de saúde; (2) os medicamentos essenciais com foco nos antimaláricos da 1.^a e 2.^a linha do protocolo MINSA; e (3) as séries temporais de stock que registam diariamente as quantidades, consumos e reposições. A estes juntam-se os registos de surtos de malária por município, permitindo correlações epidemiológicas valiosas para os decisores de saúde pública.

Padrões de Acesso (Query-Driven Design)

A modelação foi inteiramente orientada pelos padrões de acesso identificados junto dos utilizadores finais (gestores distritais, farmacêuticos, equipa DPS). A tabela seguinte lista as queries principais com estimativas de frequência e criticidade:

#	Padrão de Acesso	Tipo	Frequência	Criticidade
QA1	Municípios com surto activo E rutura de antimalárico	Correlação	Diária	Alta
QA02	Evolução mensal do stock de Coartem por município	Série Temporal	Semanal	Alta
QA03	Unidades com stock disponível mais próximas de um surto	Geoespacial	Tempo real	Crítica
QA04	Registar evento de reposição de stock	Escrita	Várias/dia	Crítica
QA05	Ranking de criticidade combinada por município	Dashboard	Diária	Alta
QA06	Eficiência logística de reposição por tipo de unidade	Análise	Mensal	Média
QA07	Unidades com múltiplos antimaláricos em rutura simultânea	Alerta	Diária	Crítica
QA08	Tendência de consumo por medicamento e época sazonal	Análise	Mensal	Média
QA09	Lista de alertas activos por município	Leitura	A cada hora	Alta
QA10	Stock actual de todos os medicamentos de uma unidade	Leitura	Múltiplas/dia	Alta

JUSTIFICAÇÃO TECNOLÓGICA

Análise Comparativa de Modelos NoSQL

Foram avaliados quatro modelos NoSQL candidatos, comparando a sua adequação aos padrões de acesso identificados. A escala de estrelas (★☆☆) representa adequação relativa a cada critério:

Critério	MongoDB	Cassandra	Redis	Neo4j
Organização dos dados	Documentos	partições e colunas	Chave-Valor	Grafos
Esquema Flexível	★★★★★	★★★★☆	★★☆☆☆	★★★★☆
Séries Temporais	★★★★☆	★★★★★	★★★★☆	★★★★☆
Geoespacial Nativo	★★★★★	★★☆☆☆	★★☆☆☆	★★☆☆☆
Aggregation / Analytics	★★★★★	★★★★☆	★★☆☆☆	★★★★☆
Correlação Epidêmica	★★★★☆	★★☆☆☆	★★☆☆☆	★★★★★
Facilidade de Modelação	★★★★★	★★★★☆	★★★★☆	★★★★☆
Curva de Aprendizagem	Suave	Íngreme	Suave	Moderada
Teorema CAP	CP/AP*	AP	CP	CA/CP
Comunidade / Documentação	★★★★★	★★★★☆	★★★★★	★★★★☆
Adequação Global ao Tema	★★★★★	★★★★☆	★★☆☆☆	★★★★☆

(*) Configurável via *write concern* e *read preference*. CP com *majority*; AP com *eventual consistency*.

Justificação da Escolha: MongoDB

Adequação ao Modelo de Dados

O domínio de monitorização de stocks de saúde pública apresenta heterogeneidade estrutural, isto é, cada medicamento possui atributos distintos (comprimidos têm dosagem em mg, kits de diagnóstico têm número de testes por caixa, mosquiteiros têm dimensões). O modelo de documentos *JSON/BSON* do MongoDB acomoda esta variabilidade sem impor migrações de esquema, que é característica essencial num sistema evolutivo que pode ser expandido para novas categorias de medicamentos ou indicadores de saúde sem downtime.

A possibilidade de embutir sub-documentos e arrays (**ex:** histórico de eventos dentro de um registo de stock, lista de medidas tomadas num surto) reduz drasticamente o número de joins necessários, acelerando as leituras. O operador **\$push** permite adicionar eventos a arrays de forma atómica, sem reescrever o documento inteiro.

Teorema CAP, PACELC e Configuração do MongoDB

O Teorema CAP (Brewer, 2000) afirma que um sistema distribuído não pode garantir simultaneamente Consistência (C), Disponibilidade (A) e Tolerância a Partições (P). Em caso de partição de rede, o sistema deve escolher entre C e A. O MongoDB, configurado com Replica Set e write concern majority, opera no quadrante CP:

Propriedade CAP	Configuração MongoDB Adoptada	Impacto no Sistema de Saúde
Consistência (C)	write concern: majority read concern: majority	Garante que nenhum alerta é gerado com dados desactualizados. Crítico para decisões clínicas sobre stocks de antimaláricos.
Disponibilidade (A)	3 nós (1P+2S): failover < 10s Replicação assíncrona	Sistema continua operacional mesmo com falha de 1 nó. Fundamental para zonas remotas do Uíge com energia instável.
Tolerância (P)	Eleição automática de novo Primary pelo protocolo Raft	Em caso de partição, o Replica Set elege novo Primary, mantendo o sistema no quadrante CP com dados íntegros.

O modelo PACELC (Abadi, 2012) estende o CAP: mesmo sem partição, há um compromisso **Latência vs. Consistência**. Para este sistema: queries de alertas críticos usam read concern majority (consistência máxima); dashboards históricos usam read preference secondaryPreferred (menor latência, aceitável para dados não críticos).

Escalabilidade: Sharding Horizontal

Embora o volume actual (166.320 documentos) não exija sharding, a arquitectura prevê a eventual partição horizontal da colecção registos_stock usando como shard key composta: { municipio_id: 1, data_registo: 1 }. Esta escolha garante que queries por município sejam roteadas a um único shard, evitando scatter-gather. Para 5 anos de dados com expansão provincial, a colecção atingirá aproximadamente 15 milhões de documentos, bem dentro da capacidade de um cluster MongoDB com 3 shards e hardware de gama média.

MODELAÇÃO DE DADOS

Arquitectura das Coleções

Coleção	N.º Docs	Estratégia	Padrão Dominante
municipios	16	Documento simples	Lookup raro - dados de referência estáticos
unidades_sanitarias	42	Ref. a municipio_id + desnorm. nome	Leitura por geolocalização (geoNear)
medicamentos	22	Documento simples	Lookup raro - catálogo de referência
registos_stock	> 166.320	Série temporal + campos desnorm	Escrita diária, leitura por range de datas
surtos_malaria	~ 80	Embedding de medidas em array	Leitura por município + período temporal
alertas	~ 529	Referências + embedding de acções	Leitura por status activo + município

Esquemas das Coleções Principais

Coleção: registos_stock (Série Temporal Principal)

Esta é a colecção com maior volume e a mais consultada. A estratégia central é a desnormalização: os nomes da unidade sanitária, município e medicamento são duplicados em cada documento, eliminando a necessidade de joins em 90% das queries. O custo adicional de armazenamento (~200 bytes/documento) é amplamente compensado pela redução de latência.

```
// Estrutura de um documento em registos_stock
{
  "_id": ObjectId("..."),
  "unidade_sanitaria_id": ObjectId("..."), // referência à colecção
  "unidade_sanitaria_nome": "Hospital Municipal de Negage", // desnorm.
  "municipio_id": ObjectId("..."),
  "municipio_nome": "Negage", // desnorm. - evita join
  "medicamento_id": ObjectId("..."),
  "medicamento_nome": "Arteméter + Lumefantrina", // desnorm.
  "categoria_medicamento": "Antimalárico",
  "relacionado_malaria": true, // flag para filtro rápido
  "data_registro": ISODate("2024-03-15"),
  "ano": 2024, "mes": 3, "semana": 11, // extraídos para $group
  "quantidade_atual": 340,
  "quantidade_entrada": 0,
  "quantidade_saida": 47,
  "stock_minimo": 400, "stock_critico": 160,
  "status": "alerta", // "normal"|"alerta"|"critico"|"rutura"
  "nivel_alerta_numerico": 1, // para $max em $group
  "dias_restantes_estimado": 7,
  "historico_eventos": [], // array de eventos (embedded)
  "registado_por": "Farmacêutico"
}
```

Coleção: municípios (com índice Geoespacial)

```
// Estrutura de um documento em municípios
{
  "_id": ObjectId("..."),
  "codigo": "MUN02",
  "nome": "Negage",
  "provincia": "Uíge",
  "populacao": 143200,
  "area_km2": 3876,
  "localizacao": {
    "type": "Point",
    "coordinates": [15.2667, -7.7667] // [longitude, latitude]
  },
  "nivel_infraestrutura_saude": 3,
  "indicadores_base": {
    "taxa_malaria_por_1000": 187.4,
    "cobertura_vacinacao_pct": 62.3,
    "distancia_media_unidade_saude_km": 12.8
  }
}
```

Estratégias de Desnormalização

Estratégia	Onde Aplicada	Optimização Resultante
Embedding de nomes (desnorm. parcial)	registos_stock, alertas	Elimina \$lookup em 90% das queries de dashboard
Flag booleana de classificação	registos_stock (relacionado_malaria)	Filtro eficiente em IXSCAN sem join com medicamentos
Campos de data extraídos	registos_stock (ano, mes, semana)	Acelera \$group por período sem operadores de data
Campo numérico de alerta (0-3)	registos_stock (nivel_alerta_numerico)	Permite \$max em \$group sem \$switch para pior status
Array de eventos embutido	registos_stock (historico_eventos)	Histórico de reposições sem coleção separada
Array de medidas embutido	surtos_malaria (medidas_tomadas)	Análise com \$unwind sem tabela de relação

Estratégia de Indexação

Coleção	Índice	Tipo	Justificação
municipios	{localizacao: '2dsphere'}	Geoespacial	Suporta \$geoNear e \$near
municipios	{codigo: 1}	Único	Lookup por código natural
unidades_sanitarias	{localizacao: '2dsphere'}	Geoespacial	geoNear - unidades próximas de surto
unidades_sanitarias	{municipio_id: 1}	Simples	Listar unidades por município
registos_stock	{municipio_id:1, data_registro:-1}	Composto	Q1, Q5: filtro + ordenação temporal
registos_stock	{medicamento_id:1, data_registro:-1}	Composto	Q2: série temporal por medicamento
registos_stock	{status:1, data_registro:-1}	Composto	Q7: filtrar por status recente

IMPLEMENTAÇÃO E RESULTADOS

Ambiente de Implementação

Componente	Especificação
SGBD	MongoDB 8.2 (imagem oficial mongo:8.2)
Cluster	Replica Set rs0: 1 Primary + 2 Secondary
Contentor	Docker 29.5.2 + Docker Compose v2
Sistema Operativo	Windows 11 (via Docker)
Linguagem Seeding	Python 3.14 + PyMongo 4.17.0
Linguagem Queries	JavaScript (mongosh)
RAM Recomendada	4 GB mínimo para MongoDB (8 GB recomendado)
Armazenamento	~2 GB para o volume de dados gerados

Volume de Dados Gerados

Coleção	Documentos	% do Total	Tamanho Estimado
municipios	16	0,01%	< 1 MB
unidades_sanitarias	42	0,03%	< 1 MB
medicamentos	22	0,01%	< 1 MB
registos_stock	166.320	99,6%	~ 800 MB
surtos_malaria	80	0,40%	~ 5 MB
alertas	529	0,2.5%	~ 15 MB
TOTAL	167.009	100%	~ 820 MB

Consultas Avançadas Implementadas

Foram implementadas 7 consultas avançadas no ficheiro *queries.js*. A tabela seguinte resume cada consulta com as técnicas MongoDB utilizadas e os tempos de execução medidos:

#	Designação	Técnicas MongoDB Utilizadas	Latência
Q1	Correlação Surto × Rutura	\$lookup com pipeline interno, \$group, \$match composto, score calculado	~1249ms
Q2	Série Temporal Mensal do Coartem	\$match de range, \$group por município+mês, \$cond para contagens condicionais	~1511ms
Q3	Geoespacial: Unidades com Stock Próximas	\$geoNear, \$lookup, \$match em resultado, \$project com cálculo km	~809ms
Q4	Atualização Atômica de Reposição	\$push com \$each/\$slice, \$set, \$inc, \$currentDate, \$match multi-critério	~561ms
Q5	Dashboard de Criticidade por Município	Duplo \$lookup com pipeline, score ponderado em \$addFields, \$switch, \$sort	~810ms
Q6	Eficiência Logística de Reposição	\$lookup, \$unwind, \$group multinível, \$cond aninhado para classificação	~794ms

Excerto de Código Q5: Dashboard de Criticidade (Pipeline de Agregação)

```
// Q5 – Ranking de Criticidade Integrada por Município
db.municipios.aggregate([
  // JOIN com stocks antimaláricos (últimos 60 dias)
  { $lookup: {
    from: "registos_stock",
    let: { mun_id: "$_id" },
    pipeline: [
      { $match: { $expr: { $eq: ["$municipio_id", "$$mun_id"] },
        relacionado_malaria: true,
        data_registro: { $gte: new Date("2024-04-01") } } },
      { $group: {
        _id: null,
        total_registos: { $sum: 1 },
        registos_rutura: {
          $sum: { $cond: [{ $eq: ["$status", "rutura"] }, 1, 0] } } } }
    ], as: "dados_stock" } },

  // JOIN com surtos de malária (2024)
  { $lookup: { from: "surtos_malaria", ... as: "dados_surtos" } },

  // Score composto ponderado (40% ruturas + 30% incidência + 20% surtos)
  { $addFields: {
    score_criticidade: { $add: [
      { $multiply: ["$_ruturas", 0.40] },
      { $multiply: ["incidencia_malaria", 1.50] },
      { $multiply: ["n_surtos", 3.0] }
    ] } } },

  { $sort: { score_criticidade: -1 } }
])
```

DISCUSSÃO CRÍTICA

Limitações da Solução Actual

- **Latência das queries de correlação (Q1, Q5):** O duplo \$lookup em registos_stock sem sharding tem complexidade $O(n)$ sobre 166.320 documentos. Para volumes maiores, será necessário materializar vistas ou usar Change Streams para manter coleções de resumo pré-computadas.
- **Ausência de Time-Series Collections nativas:** O MongoDB 5 + introduziu coleções optimizadas para séries temporais. A migração de registos_stock para este tipo reduziria o armazenamento em ~40% e aceleraria range queries temporais.
- **Sem validação de esquema aplicada:** Em produção é essencial definir JSON Schema Validation na colecção registos_stock para garantir integridade dos campos obrigatórios (status, quantidade_atual, data_registo).
- **Replica Set vs Sharded Cluster:** A configuração actual (3 nós) não escala horizontalmente o armazenamento — apenas a leitura. Para múltiplas províncias angolanas seria necessário migrar para Sharded Cluster.

Escalabilidade: Cenário de Volume 100x

Se o volume de dados aumentasse 100x (expansão para as 18 províncias estudadas, registo de 5 em 5 minutos em vez de diário), a colecção registos_stock atingiria ~21 milhões de documentos e ~82 GB:

Problema	Impacto	Solução MongoDB
registos_stock > 20M docs	Q1, Q5 degradam para > 10s	Sharding: shard key {municipio_id, data_registo}
WiredTiger cache miss	Latência sobe para 2-5s	Aumentar RAM ou Atlas Search para queries complexas
Índices em disco > 2 GB	Escritas mais lentas	Sparse indexes; TTL index para dados > 2 anos
Replica Set lag	Secondaries atrasados	Aumentar oplog size; otimizar write concern
Single-region	Falha catastrófica	MongoDB Atlas Multi-Region (ex: Africa/Lagos)

Comportamento em Falha de Rede (Partição)

O Replica Set garante a seguinte sequência em caso de falha do nó Primary:

- **$t = 0s$:** O nó Primary (mongo-primary) fica inacessível por falha de hardware ou partição de rede.
- **$t = 0-10s$:** Os dois Secondary nodes detectam ausência de heartbeat e iniciam eleição automática pelo protocolo Raft.
- **$t \approx 10s$:** Um Secondary é eleito novo Primary. O write concern majority funciona com os 2 nós restantes.

- ***t = 10s+***: Sistema operacional. Escritas e leituras servidas pelo novo Primary sem intervenção manual.
- ***Recuperação***: Quando o nó original volta, sincroniza automaticamente via oplog replay e retoma o papel de Secondary.
- ***Cenário de risco***: Se dois dos três nós ficarem inacessíveis simultaneamente, o único nó restante recusa escritas (prioriza Consistência sobre Disponibilidade, comportamento CP esperado e desejado para dados de saúde pública).

Melhorias Futuras

- Implementar Change Streams para alimentar alertas em tempo real sem polling.
- Migrar registros_stock para Time-Series Collection nativa (MongoDB 5+).
- Adicionar Atlas Search para pesquisa textual sobre nomes de medicamentos e municípios.
- Integrar com DHIS2 (sistema de informação de saúde usado pelo MINSA) via API REST.
- Implementar encriptação at-rest (AES-256) e TLS para dados sensíveis de saúde.
- Dashboard Kibana ou Grafana conectado ao MongoDB para visualização em tempo real.

REFERÊNCIAS BIBLIOGRÁFICAS

- [1] E. Brewer, "Towards robust distributed systems" (palestra tônica), in Proc. ACM Symp. Principles of Distributed Computing (PODC), Portland, OR, EUA, 2000.
- [2] D. Abadi, "Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story," IEEE Computer, vol. 45, n.º 2, pp. 37-42, Fev. 2012, doi: 10.1109/MC.2012.33.
- [3] K. Chodorow, MongoDB: The Definitive Guide, 3.ª ed. Sebastopol, CA: O'Reilly Media, 2019.
- [4] MongoDB, Inc., "MongoDB 7.0 Manual - Replication," MongoDB Documentação, 2024. Disponível em: <https://www.mongodb.com/docs/v7.0/replication/>
- [5] MongoDB, Inc., "Aggregation Pipeline Reference," MongoDB Documentation, 2024. [Online]. Disponível em: <https://www.mongodb.com/docs/manual/reference/operator/aggregation-pipeline/>
- [6] MongoDB, Inc., "Geospatial Queries," MongoDB Documentation, 2024. [Online]. Disponível em: <https://www.mongodb.com/docs/manual/geospatial-queries/>
- [7] R. Cattell, "Scalable SQL and NoSQL Data Stores," ACM SIGMOD Record, vol. 39, n.º 4, pp. 12-27, Dez. 2010, doi: 10.1145/1978915.1978919.
- [8] M. Fowler e P. Sadalage, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Upper Saddle River, NJ: Addison-Wesley, 2012.
- [9] Ministério da Saúde de Angola (MINSa), Programa Nacional de Controlo da Malária — Guia de Tratamento da Malária, 4.ª ed. Luanda: MINSa, 2021.
- [10] Organização Mundial da Saúde (OMS), Guidelines for the Treatment of Malaria, 3.ª ed. Genebra: WHO Press, 2015.
- [11] Instituto Nacional de Estatística de Angola (INE), Resultados Definitivos do Recenseamento Geral da População — Província do Uíge. Luanda: INE, 2016.
- [12] J. Han, E. Haihong, G. Le, e J. Du, "Survey on NoSQL database," in Proc. 6th Int. Conf. Pervasive Computing and Applications (ICPCA), 2011, pp. 363-366, doi: 10.1109/ICPCA.2011.6106531.
- [13] A. Lakshman e P. Malik, "Cassandra: A Decentralized Structured Storage System," ACM SIGOPS Oper. Syst. Rev., vol. 44, n.º 2, pp. 35-40, Abr. 2010, doi: 10.1145/1773912.1773922. [14] P. Membrey, D. Plugge, and T. Hawkins, The Definitive Guide to MongoDB, 3rd ed. New York, NY: Apress, 2015, doi: 10.1007/978-1-4842-1183-0.

ANEXO A (GUIA DE INSTALAÇÃO)

A.1 Instalação via Docker (Recomendada)

```
# 1. Verificar instalação do Docker
docker --version && docker compose version

# 2. Clonar o repositório
git clone https://github.com/jozzoficial/monitorizacao-saude-uige.git
cd nosql-saude

# 3. Iniciar cluster MongoDB (1 Primary + 2 Secondary)
docker-compose up -d

# 4. Aguardar ~30 segundos e verificar o Replica Set
docker exec -it mongo-primary mongosh --eval "rs.status()"

# 5. Instalar dependências Python
pip install pymongo faker

# 6. Povoar a base de dados (gera > 177.009 documentos)
python seed.py

# 7. Executar as 7 queries avançadas
mongosh saude_uige --file queries.js
```

A.2 Instalação Local — Linux (Ubuntu 22.04)

```
# Importar chave GPG do MongoDB
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | \
  sudo gpg --dearmor -o /etc/apt/trusted.gpg.d/mongodb-server-7.0.gpg

# Adicionar repositório
echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu \
  jammy/mongodb-org/7.0 multiverse" | \
  sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list

# Instalar, iniciar e habilitar o serviço
sudo apt-get update && sudo apt-get install -y mongodb-org
sudo systemctl start mongod && sudo systemctl enable mongod

# Conectar ao shell
mongosh
```


A.3 Verificações Rápidas no mongosh

```
// Verificar contagens por coleção
use saude_uige
db.getCollectionNames().forEach(c =>
  print(c.padEnd(25) + ": " + db[c].countDocuments() + " docs"))

// Testar índice geoespacial (unidades próximas de Uíge sede)
db.unidades_sanitarias.find({
  localizacao: {
    $near: {
      $geometry: { type: "Point", coordinates: [15.0581, -7.6094] },
      $maxDistance: 50000 // 50 km
    }
  }
}, { nome: 1, tipo: 1, municipio_nome: 1, _id: 0 }).limit(5)

// Ver estado do Replica Set
rs.status()
```